NARASARAOPETA ENGINEERING COLLEGE (AUTONOMOUS)

Official Study Material • Java Programming & OOP (JAVA101) • Unit 2: Classes, Objects & Inheritance

Course Code:	JAVA101	**Regulation:**	R20 / R23 Autonomous
Subject:	Java Programming & OOP	**Unit:**	Unit 2 - Classes, Objects & Inheritance

NARASARAOPETA ENGINEERING COLLEGE (AUTONOMOUS)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ACADEMIC STUDY MATERIAL: JAVA101 JAVA PROGRAMMING & OOP

UNIT 2: CLASSES, OBJECTS & INHERITANCE

Regulation: R20 / R23 Autonomous Syllabus | Credits: 4

1. CLASSES, OBJECTS & CONSTRUCTORS:

- Class: Blueprint or template defining states (variables) and behaviors (methods).

- Object: Real-world instance of a class occupying heap memory.

- Constructors: Special methods invoked during object creation with new keyword. Same name as class, no return type.

* Default Constructor, Parameterized Constructor, and Constructor Overloading.

- 'this' Keyword: Reference variable referring to current invoking object (resolves namespace collisions: this.id = id).

- Garbage Collection: Automatic memory management in JVM that destroys unreferenced heap objects via mark-and-sweep.

2. INHERITANCE IN JAVA:

Inheritance establishes an IS-A relationship between parent (superclass) and child (subclass) using extends keyword.

- Supported Types: Single Inheritance, Multilevel Inheritance, Hierarchical Inheritance.

- Multiple Inheritance of classes is prohibited in Java to avoid the Diamond Problem (ambiguity). It is achieved using Interfaces.

- Method Overriding: When child class defines a method with identical signature and return type as parent class. Represents Runtime (Dynamic Method Dispatch) Polymorphism.

- 'super' Keyword: Refers to immediate parent class (invokes parent constructor super(), accesses parent fields and methods).

- 'final' Keyword:

* final variable: Constant value; cannot be reassigned.

* final method: Cannot be overridden by child class.

* final class: Cannot be inherited (extended).

3. ABSTRACT CLASSES VS INTERFACES:

- Abstract Class (abstract keyword): Can have abstract methods (no body) and concrete methods with state. Can have constructors.

- Interface (interface keyword): Contract of methods. All methods are public abstract by default. Since Java 8, interfaces can contain default and static methods. Since Java 9, private methods. Implemented with implements keyword.

4. PRACTICAL CODE EXAMPLE:

```
// Java Program demonstrating Inheritance, Method Overriding, super, and Interfaces
interface Printable {
void printDetails();
```

```
}

class Employee implements Printable {
protected int empId;
protected String name;

public Employee(int empId, String name) {
this.empId = empId;
this.name = name;
}

public double calculateSalary() {
return 30000.0;
}

@Override
public void printDetails() {
System.out.println("Employee ID: " + empId + ", Name: " + name);
}
}

class Manager extends Employee {
private double bonus;

public Manager(int empId, String name, double bonus) {
super(empId, name); // Call superclass constructor
this.bonus = bonus;
}

@Override
public double calculateSalary() {
return super.calculateSalary() + bonus; // Method Overriding
}

@Override
public void printDetails() {
super.printDetails();
System.out.println("Role: Manager | Total Salary: " + calculateSalary());
}
}

public class InheritanceDemo {
public static void main(String[] args) {
Employee emp = new Manager(101, "Ravi Kumar", 25000.0); // Dynamic dispatch
emp.printDetails();
}
}
```

Output:

Employee ID: 101, Name: Ravi Kumar

Role: Manager | Total Salary: 55000.0

5. IMPORTANT EXAM QUESTIONS & ANSWERS:

Q1: Differentiate between Method Overloading and Method Overriding.

Answer: Overloading occurs in the same class with different parameters (compile-time polymorphism). Overriding occurs between superclass and subclass with identical signatures (runtime polymorphism). Return type alone is insufficient for overloading.

Q2: Compare Abstract Classes and Interfaces in Java.

Answer: Abstract classes can have instance variables and concrete methods, supporting single inheritance. Interfaces cannot hold instance state (only public static final constants) and allow multiple inheritance of type.